Reinforcement Learning for Congestion Control

A Review of the 50 Most-Cited Studies

Gaurab Baral Nityashree Vangala

CS 6043: Networking
Fall 2026

Abstract

Reinforcement learning (RL) lets a congestion controller learn its control law from experience. Our searches identified 2,459 records and 362 primary studies of RL and imitation learning for congestion control published from 2016 to 2026. Because of time constraints, this report reviews the 50 of those studies with the most citations in a single Semantic Scholar snapshot taken on 9 November 2026. We ask how these controllers are designed, how they are tested, and what keeps operators from using them. Of the 43 design studies, 33 run at the sender and 15 keep a classic algorithm or safeguard in the deployed control loop. Of the 42 studies whose evaluation level we could determine, 18 (43%) reach a real network. Only 7 design studies test outside their training distribution and only 7 release code. Five report production use or tests with real users. Four of those keep a classic component in the loop, and the fifth learned from logged sessions. Citation ranking favors older work: only 4 of the 50 studies first appeared in 2023 or later, against 70% of the remaining studies. Our findings therefore describe the literature's most influential papers and should not be read as a census of current practice.

1 Introduction

1.1 Motivation

Congestion control (CC) decides how fast a sender may send its data into a shared network. It is one of the oldest control problems in networking. After major Internet congestion in 1986, Jacobson's slow start and additive-increase/multiplicative-decrease (AIMD) methods helped stabilize the network by adjusting sending rates when packets were lost [1, 2]. Almost four decades later the problem takes many forms. Today, data travels over gigabit fiber, shared Wi-Fi, cellular networks whose capacity changes every few milliseconds, satellite networks that frequently switch connections between satellites [3, 4], and data-center networks where a round trip takes only tens of microseconds. Despite these advances, congestion remains difficult to manage. Some challenges in CC include packet loss unrelated to congestion, noisy delay measurements, and widely varying buffer sizes [5, 6]. These challenges have led researchers to design algorithms for specific environments: Vegas, CUBIC, BBR and Copa for the Internet, Sprout, Verus and ABC for wireless networks, and DCTCP, DCQCN, TIMELY, HPCC, Swift, PowerTCP and Bolt for data centers [7, 8].

With reinforcement learning (RL), a controller learns its control law instead of having it written by hand. In RL-based congestion control, an agent at the sender (or in the network) watches signals such as delay, throughput, and packet loss. Every short interval this agent raises or lowers the sending rate or the congestion window, and it receives a reward that is high for fast delivery and low for long delays or lost packets. Over many trials it learns which action works best in each network state. Remy [9] and PCC [10, 11] showed that machine-generated or online-learned controllers can beat hand-tuned ones. Aurora [12] then cast Internet congestion control explicitly as a deep RL problem in 2018–2019, and the field has grown quickly since.

Our searches retrieved 2,459 records, and 362 primary studies met our eligibility criteria (Section 1.6). However, learned controllers that look excellent in a simulator sometimes fail on network links they never saw. Few of them run inside an operating-system kernel, and fewer still have been deployed by an operator. Due to time constraints, we examine only the 50 studies with the most citations.

1.2 Research questions and contributions

We organize the survey around three research questions.

RQ1 **Design.** How do RL methods control congestion: what do they observe, what do they change, and what reward do they learn from?

RQ2 **Testing.** How are these methods trained and tested, and do they still work on networks they did not see in training?

RQ3 **Deployment.** What stops operators from using RL-based congestion control in real networks, and what research could fix these problems?

This report contributes a reproducible ranking of the 362 eligible studies by citations, a design and evidence profile of the top 50 (Sections 2 and 3, Section B), a comparison of those 50 with the other 312 (Table 2), and an analysis of the barriers to deployment that the 50 expose (Section 4).

1.3 Congestion control

A congestion controller should use the available bottleneck capacity, avoid standing queues and loss, share capacity among competing flows, converge quickly when conditions change, and coexist with the deployed base, which in practice means not starving CUBIC and BBR flows. Fairness is often measured with Jain's index [13]. Ware et al. argue that deployability should instead be judged by the *harm* a new algorithm does to existing traffic [14].

End-to-end controllers infer congestion from what they can measure: packet loss, round-trip time (RTT) and its trend, delivery rate, and Explicit Congestion Notification (ECN) marks set by routers [15]. They act through the *congestion window* (cwnd), which caps the data in flight, or the *pacing rate*, which spaces packets in time.

Reno halves cwnd on loss and grows it by one packet per RTT [16]. CUBIC, the Linux default, grows cwnd as a cubic function of the time since the last loss [17]. Vegas and Copa back off when the RTT rises above its minimum [18, 19]. BBR estimates the bottleneck bandwidth and minimum RTT and paces at the estimated bandwidth [20]. Routers and switches take part through active queue management (AQM), which drops or marks packets before the buffer fills, as in RED, CoDel and PIE [21–23]. In data centers, DCTCP reacts to the fraction of ECN-marked packets [24], and its performance depends on a marking threshold that operators usually tune by hand.

1.4 Reinforcement learning

RL formalizes sequential decision making as a Markov decision process: at each step the agent observes a state s_t, takes an action a_t from its policy π, receives a reward r_t, and the environment moves to s_{t+1}. The goal is a policy that maximizes the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ [25]. A sender sees only its own feedback, so congestion control is formally partially observable [26]. Most designs compensate by feeding the agent a short history of observations. The 50 studies use the following families of methods.

- **Value-based methods** learn the action-value function $Q(s, a)$. Tabular Q-learning suits small discrete state spaces, and the Deep Q-Network (DQN) [27] approximates Q with a neural network.

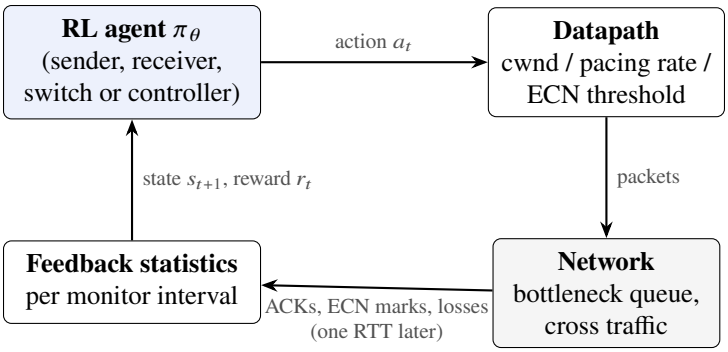


Figure 1: The usual RL congestion-control loop. The agent acts once per monitor interval and observes the consequences one RTT later.

- **Policy-gradient methods** optimize π directly. PPO [28] and A3C [29] handle continuous rate multipliers naturally.
- **Off-policy actor-critic methods** such as DDPG [30] and TD3 [31] reuse past experience from a replay buffer. IMPALA [32] decouples acting from learning.
- **Offline RL** learns from a fixed dataset of logged interactions without exploring online [33].
- **Imitation learning** trains a policy to copy an expert, which can be an oracle with full knowledge of a simulated link or an existing heuristic [34].
- **Bandits** treat each decision interval as a single-step problem.
- **Multi-agent RL** trains several interacting agents, for example competing flows or switches [35].
- **Multi-objective RL** learns one policy for many trade-offs [36].

1.5 Congestion control as an RL problem

Figure 1 shows the formulation that Aurora made popular [12]. Time is divided into *monitor intervals*. The agent picks an action at the start of each interval, and when the ACKs for that interval return, the sender computes statistics and the reward. Designs vary in five respects.

1. **Placement.** Most agents run at the sender. Some run at the receiver, which is common for real-time video. Others sit in switches and routers or in a central controller.
2. **Observation.** Typical inputs are a short history of throughput, RTT, RTT gradient, loss rate and the current cwnd or rate. Aurora chose scale-free features (latency gradient, latency ratio and sending ratio) to help generalization across link speeds.
3. **Action.** The action can set the cwnd or rate, tune a parameter of a classic algorithm, select among existing algorithms, or set a network parameter such as an ECN threshold.
4. **Objective.** Most rewards combine throughput, delay and loss. Aurora uses $10 \cdot \text{throughput} - 1000 \cdot \text{latency} - 2000 \cdot \text{loss}$. The reward fixes the operating point, so it acts as the controller's specification.
5. **Decision frequency.** Neural inference on every ACK is too expensive at line rate, so most designs act once per interval.

The consequences of an action arrive one RTT later and mix with the effects of other flows, so credit assignment is noisy. A controller also runs on networks whose statistics differ from any training environment, which makes performance under distribution shift a central measure of quality.

1.6 Review method

The protocol query was

```
("reinforcement learning" OR "imitation learning") AND
("congestion control" OR "congestion window" OR "ECN")
```

applied to titles and abstracts of works published 2016–2026. The database and citation searches were run on 31 October 2026, and an update search with broadened terms on 9 November 2026. Search strings in each source's syntax are given in Section A.

We followed the PRISMA 2020 reporting guideline [37], with the search and snowballing steps adapted to computer science as recommended by Kitchenham and Charters [38] and Wohlin [39]. The whole pipeline is in the replication code file that we submit with this survey.

Eligibility. We included primary studies, in English, in which an RL or imitation-learning method (including bandits, offline RL and multi-agent RL) controls congestion in a packet network, along with studies that evaluate such controllers and frameworks built to train or test them. We excluded congestion outside packet networks, RL for other networking tasks such as routing or adaptive-bitrate video over TCP, methods with no RL component, and secondary studies.

Selection of the 50. Figure 2 shows the flow. Reviewing the top 50 is a deviation from our protocol, which covered every eligible study. For each of the 362 studies we retrieved citation counts from the Semantic Scholar API on 9 November 2026, so that all counts come from one source on one date. Lookups used the DOI or arXiv identifier, or an exact title match when neither resolved. When a study has several linked reports, such as a conference paper and its journal extension, we used the highest count among them and did not add the counts, because such reports can share citing papers. Ties were broken by earlier publication year and then by study identifier. The cutoff is 27 citations, and 4 studies are tied at that value, so the tie-break decided two of the last places. Semantic Scholar had no record for 9 studies. We checked them in OpenAlex, where none has more than seven citations, so none could have reached the cutoff.

Data items and appraisal. For each study we coded the setting, control point, RL method, state, action, reward and training setup. We also coded the highest evaluation level reached: *simulation* (a discrete-event or fluid model), *emulation* (real protocol stacks over emulated links) or *real network* (real links or production traffic). Finally, we coded whether generalization outside the training range was tested, whether fairness was evaluated, and whether code was released. We call a design *hybrid* when a classic algorithm or safeguard is present in the deployed control loop. Each design study receives a *practice score* from 0 to 5, with one point each for reporting the evaluation environment, using emulation or a real network, testing generalization, evaluating fairness, and releasing code. Of the 50 studies, 19 were coded from full texts, 29 from abstracts and metadata, and 2 from the reviewers' knowledge of the paper. Scores for abstract-coded studies are lower bounds. We synthesize narratively and do not pool effect sizes, because the studies use incomparable baselines, links and metrics.

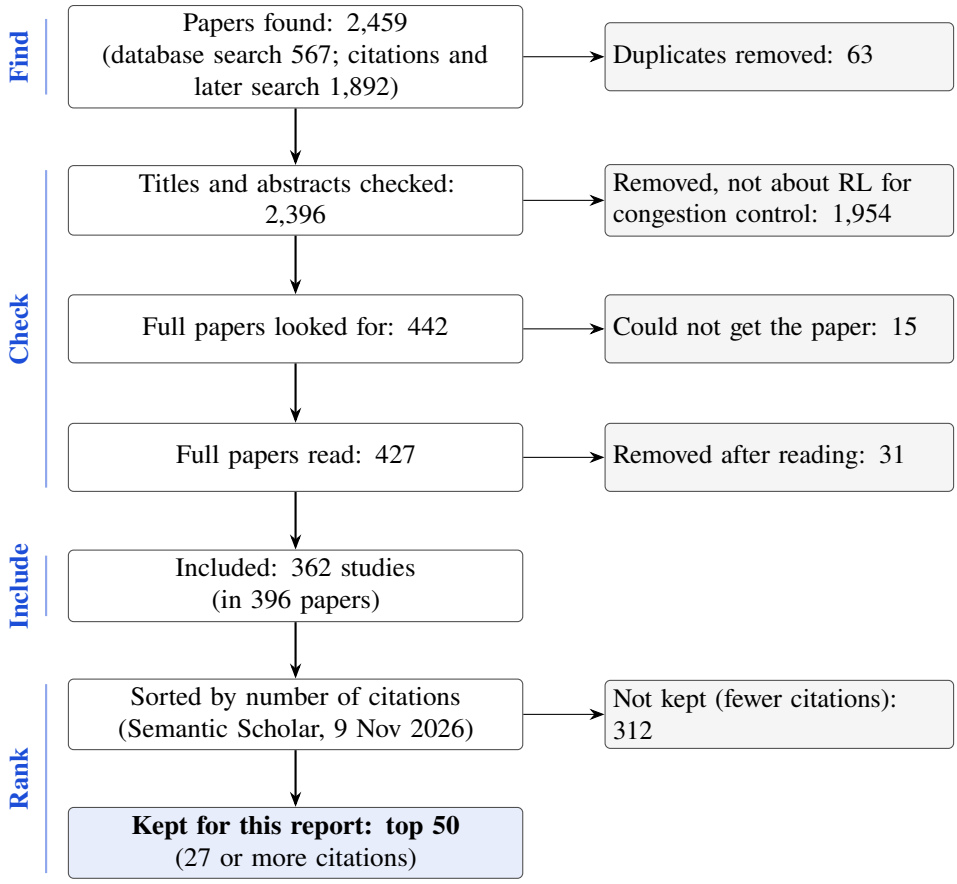


Figure 2: How the 50 studies were chosen, drawn in the style of a PRISMA 2020 flow diagram [37]. The top part is the search behind the full survey. The last step, which keeps the 50 most-cited studies, is new in this report. Some studies appear in more than one paper, such as a conference paper and its journal version, so 396 papers make up 362 studies.

1.7 Related surveys

Broad surveys of machine learning or deep RL for networking treat congestion control as one application among many [40–42]. Magazine-style overviews discuss a handful of prominent designs and the challenges of standardizing them [43–45]. Jiang et al. give the most detailed pre-2021 summary and add an experimental comparison [46]. Later reviews narrow the scope to wired networks [47], multipath TCP [48], active queue management [49] or fairness [50]. The two most recent are RL-specific [51, 52], and Abedini et al. compare RL algorithms, state, action and reward definitions, simulation platforms and outcomes. Vielhaus et al. survey how congestion-control algorithms in general are evaluated [53]. This report differs in three ways: its studies come from a scripted search with released decisions, it records the evaluation level of every study, and it compares the most-cited studies with the rest of the eligible literature.

2 RL congestion control for the wide-area network

Of the 50 studies, 36 address the wide-area Internet or a generic wired bottleneck. Of the others, 6 address wireless or cellular networks, 4 IoT or sensor networks, 3 data centers and 1 satellite links. This section follows the wide-area designs and closes with the 14 studies from other settings (Section 2.9), so that every reviewed study is covered. Studies that build frameworks or analyze controllers are discussed in Section 3. Table 1 summarizes the designs that later work most often builds on.

Table 1: Landmark designs among the 50. Details come from the full texts except where marked † (abstract and reviewers' knowledge). NR: not reported in the material we could access. Level: highest evaluation environment.

Study	Action	Observation	Learning signal	Algorithm	Level
Indigo [54]	cwnd step every 10 ms	averages of queueing delay, sending and receiving rate; cwnd; previous action	imitation of an oracle that knows the emulated link's ideal cwnd	IL, LSTM	Real
Aurora [12]	rate multiplier per interval	history of latency gradient, latency ratio, sending ratio	10 thr − 1000 lat − 2000 loss	PPO	Emu
MVFST-RL [55]	cwnd ∈ $\{\cdot, /2, -10, +10, \times 2\}$	ACK statistics over 100 ms plus action history	throughput–delay trade-off	IMPALA	Emu
Orca [56]	cwnd = $2^{2a} \cdot$ CUBIC's cwnd	throughput, delay, loss statistics	throughput minus loss penalty, scaled by a delay factor	TD3	Real
Eagle† [57]	rate	network statistics with LSTM memory	reward plus guidance from BBR as expert	DRL + IL	Emu
Owl [58]	cwnd (Linux kernel module)	end-to-end statistics plus in-network signals	a utility function	DQN	Emu
MOCC [59]	rate	network statistics plus objective weights	preference-weighted throughput, delay, loss	PPO + preference sub-net	Real
Genet [60]	(training method)	Aurora's	Aurora's	PPO + curriculum	Real
Sage† [61]	cwnd	general representation of many path statistics	logged behavior of existing heuristics	offline RL	Real
Spine† [62]	sub-policy for a light CC executor	NR	NR	hierarchical DRL	Real
Astraea [63]	rate	local, global and historical statistics	throughput, delay, loss, fairness	multi-agent actor-critic	Real
Mutant [64]	select a kernel CC scheme	raw transport statistics	power (throughput over delay)	LinUCB bandit	Real

Across the 43 design studies, 33 run the agent at the sender. The action sets the cwnd in 13, a sending rate in 12, a media bitrate in 6, a parameter or algorithm choice in 8, and a network-side value (an AQM decision or ECN threshold) in 4.

2.1 Early Q-learning controllers

The earliest designs among the 50 use Q-learning with simple function approximation. QTCP approximates the Q-function with generalized Kanerva coding and reports 59.5% higher throughput than rule-based TCP [65]. RL-TCP learns cwnd adjustments for shallow buffers and improves on NewReno's throughput–delay trade-off in ns-2 [66]. These designs have small, hand-quantized state spaces.

2.2 Deep RL and clean-slate policies

Aurora [12] set the formulation most later work adopts. Trained with PPO on simulated links of 1.2–6 Mbit/s with 50–500 ms latency and 0–5% random loss, it kept near-full utilization in emulation from 1 to 128 Mbit/s and compared well with CUBIC and PCC Vivace. It performed poorly at 1 ms latency, where about 1 ms of packet-processing noise absent from training dominated the latency signal. Its authors named fairness, safety and generalization as open problems.

Indigo [54], released with the Pantheon testbed, trained an LSTM by imitating an oracle that knew the ideal window of each emulated link. MVFST-RL [55] addressed a problem that synchronous simulators hide: in a real stack, the network does not pause while the agent computes. It runs the policy asynchronously inside a QUIC implementation and trains with IMPALA. TCP-Drinc [67] feeds a history of measurements to a DQN with convolutional and LSTM layers and outperforms five benchmark schemes in ns-3. Owl [58]

adds in-network signals to end-to-end statistics and runs DQN in a Linux kernel module. It converges to fair allocations on emulated and large virtual testbeds. Eagle [57] uses BBR as a teacher during training, and it matches or beats that teacher.

2.3 Hybrids that learn on top of classic control

Hybrids make up 15 of the 43 design studies (35%). They let a classic algorithm react quickly and let the agent adjust it more slowly. Orca [56] is the reference design: CUBIC runs per ACK in the kernel, and once per monitoring period a TD3 agent multiplies CUBIC's window by 2^{2a}, $a \in [-1, 1]$. Evaluated on emulated links and global testbeds, it performed more consistently than clean-slate learned controllers at lower overhead. Spine [62] has a coarse DRL agent emit sub-policies for a light executor, which cuts control overhead by at least 80% relative to DRL controllers. In Libra [68], a classic algorithm and an RL policy each propose a rate, and the sender commits to whichever yields higher utility in a brief evaluation. It reports 1.2 times Orca's throughput on dynamic networks.

Two designs learn *which* algorithm or setting to use. TCP-RL [69] uses group-based RL to set the initial window and deep RL to select among 14 algorithms, and it ran on the web servers of a top global search engine for more than a year. Mutant [64] uses a LinUCB contextual bandit to switch among kernel protocols in real time, with negligible harm to competing flows.

2.4 Learning from logs

Sage [61] trains offline on trajectories from a pool of existing heuristics, and in Internet and controlled tests it learned a policy that performs better than the heuristics it observed. Concerto [70] applied imitation learning to more than one million logged video sessions and was incorporated into a production video-telephony application.

2.5 Multiple objectives and fairness

A single reward fixes one operating point. MOCC [59] conditions a PPO policy on a preference vector so that one model competes with the best specialized algorithm for each objective, and transfer learning lets it adapt to new objectives 14.2 times faster than prior work. A policy trained alone also tends to be greedy when copies of it share a bottleneck. Astraea [63] trains flows jointly with a fairness term in the reward and reports near-optimal fairness with up to 8.4 times faster convergence.

2.6 Multipath transport

Multipath TCP adds a second decision, how to divide traffic over subflows, and a constraint against taking more than a single-path flow at a shared bottleneck. DRL-CC [71] uses an LSTM actor-critic over all subflows and remains TCP-friendly. DQL-MPTCP [72] combines deep Q-learning with policy gradients and beats LIA, OLIA and BALIA in emulation.

2.7 Real-time media

In interactive video, the controller is a bandwidth estimator whose output sets the encoder's target bitrate, and rewards are quality-of-experience terms. QARC [73] paired A3C with a video-quality predictor in trace-driven simulation. Fang et al. [74] trained PPO for real calls and documented a gap between simulated and real reward. Loki [75] fuses an RL model with a reverse-engineered rule-based model and was evaluated in a commercial service over 101 million sessions, reducing stall rate by 26.3–44.2% against rule-based and

learning-based baselines. Two further designs combine learning with a heuristic: HRCC [76] lets an agent set a gain on a heuristic estimate, and BoB [77] warm-starts PPO with a heuristic.

2.8 Other transports and network-side control

Named Data Networking changes the control point, since consumers pace their requests. IEACC [78] uses edge-aided congestion degrees from routers and reports higher rates and lower loss than other NDN schemes in simulation. In software-defined networks, Lei et al. train a central agent on congestion control with load balancing as an auxiliary task [79]. Sert et al. learn an update rate that lowers age of information over TCP/IP in emulation [80].

Three studies place learning in the router. QRED uses Q-learning to tune RED's maximum drop probability and keeps the queue more stable than RED in ns-2 [81]. Gómez et al. combine Q-learning with a neural congestion predictor and evaluate an ECN-based AQM in Mininet [82]. Ma et al. apply deep RL to AQM [83], but we could code that study only from its title and metadata.

2.9 Other network settings

Wireless and cellular. Abbasloo et al.'s DeepCC is a DDPG plug-in that caps the window of existing TCP schemes, steering CUBIC, BBR, Westwood and Illinois to target delays on cellular links [84]. OnRL [85] learns online in a mobile video service. Each user runs an agent, the agents' experience is aggregated into one model, and a rule-based safeguard takes over when the policy misbehaves. Over 543 hours of sessions from 151 users it reduced stalling by 14.2% at similar quality. SmartCC [86] and He et al.'s DeepCC [87] address multipath TCP in heterogeneous wireless networks, and Xie et al. set the initial window for 5G edge computing with deep RL in ns-3 [88]. Hd-TCP adapts to high-speed railway conditions and is evaluated in simulation on field-measured conditions [89].

IoT and sensor networks. Li et al. use Q-learning with function approximation to reduce memory on constrained devices and report 33.8% higher throughput and 12.1% lower delay than NewReno in simulation [90]. iCoCoA applies deep RL to congestion control for CoAP [91]. Naeem et al. run a central actor-critic for multipath traffic in software-defined IoT networks [92], and Kim et al. use DQN for AQM in fog and edge networks [93]. All of the IoT studies with a known evaluation level stop at simulation.

Satellite. AUTO [94] uses multi-objective RL with a preference-adaptation model for satellite-ground networks and evaluates it in emulation and on real Internet paths.

Data centers. Operators control the switches, and a round trip takes only tens of microseconds. ACC [95] runs an agent on every switch to adjust ECN thresholds at run time, and in production environments of 300 machines it achieved up to 20% higher storage IOPS and 30% lower flow completion time than tuned static thresholds. RL-CC [96] treats RDMA congestion control as a multi-agent problem, and in simulation it beat RL baselines and production algorithms and generalized to unseen scenarios. Iroko [97] is covered in Section 3.4.

Summary for RQ1. The most-cited designs observe short histories of throughput, delay and loss, act on the window or the rate, and optimize a weighted reward. The later ones among them increasingly bound the learned part with a classic algorithm or learn it from data.

Table 2: The 50 most-cited studies against the other 312 eligible studies. Percentages of generalization, fairness, code and hybrid use design studies as the denominator. Evaluation shares use studies whose level is known.

	Top 50	Other 312
Design studies	43	282
Reach a real network (of known level)	18/42 (43%)	39/204 (19%)
Simulation only (of known level)	13/42 (31%)	131/204 (64%)
Level not reported	6/48	106/310
Test outside training distribution	7 (16%)	28 (10%)
Evaluate fairness	12 (28%)	51 (18%)
Release code	7 (16%)	18 (6%)
Hybrid	15 (35%)	78 (28%)
First reported 2023 or later	4 (8%)	218 (70%)
Coded from full text	19 (38%)	62 (20%)
Median practice score (0–5)	2	1

3 Training and testing: from simulators to real networks

3.1 The evaluation ladder

We distinguish four rungs by how much of the real system they include. *Simulation* models both network and endpoints, using packet-level tools such as ns-2, ns-3 and OMNeT++ or custom models. It leaves out the operating system's stack, including timers, pacing and ACK aggregation. *Emulation* runs real stacks over links imposed in software, with tools such as Mininet [98], Mahimahi [99] and NetEm. *Shared and hardware testbeds* put real stacks on real links under controlled conditions, as Pantheon's nodes do [54]. *Real networks and production* expose the controller to real paths and traffic, through experiments between cloud regions, A/B tests or full deployment. We count testbeds as real network when the links are physical. Our project proposal called Wireshark a simulator, which was a mistake: Wireshark is a packet analyzer and can be used on any of these rungs.

3.2 Where the 50 stand

Table 2 compares the 50 with the other 312 eligible studies. Of the 48 studies with an empirical evaluation, 18 reach a real network, 11 stop at emulation, 13 stop at simulation, and 6 do not report a level in the material we could access. The remaining 2 studies are formal verification.

The most-cited studies reach real networks more than twice as often and rely on simulation alone about half as often. We cannot attribute this gap to citations alone. The top 50 were coded from full texts about twice as often, and abstracts omit details, so part of the difference comes from how much we could read. The top group is also older and concentrated at prominent venues.

Citations do not guarantee strong evidence. The median design study in the top 50 satisfies only 2 of the five appraised practices, and only 6 satisfy four or more: Orca satisfies all five, and Aurora, Genet, MOCC, Astraea and Mutant satisfy four. QTCP, ranked fourth, scores zero, although that score reflects a study we could code only from its abstract and is therefore a lower bound.

3.3 Evaluation practices

Only 7 of the 43 design studies test explicitly outside their training range: Aurora, Orca, Genet, OnRL, MOCC, RL-CC and Astraea. Fairness is evaluated in 12, and code is released by 7: Aurora, Orca, Genet, Sage, Abbasloo et al.'s DeepCC, BoB and Mutant. Limited code release narrows what later work can compare

against. In the full texts we read, comparisons with learned baselines tend to involve the few controllers with public code. Deep RL results also vary with seeds and implementation details [100], and we could not code seed reporting from abstracts, so we make no claim about it.

3.4 Frameworks and analyses

Four of the 50 are frameworks. Pantheon [54] offers a community testbed with emulators calibrated against real paths. MVFST-RL provides an asynchronous training environment for QUIC. OpenNetLab [101] combines a gym environment with distributed real nodes for real-time media. Iroko [97] emulates data-center topologies for PPO, REINFORCE and DDPG agents, and its authors report instability and overfitting to the training topology.

Three are analyses. Verily [102] and whiRL 2.0 [103] formally verify trained policies, including Aurora, and whiRL proves some requirements while finding counterexamples for others. Metis [104] converts deep-learning networking systems into decision trees. Aurora appears in Metis as an example of the end-host congestion-control systems the method applies to, and Metis is therefore the most loosely connected member of the 50.

3.5 Generalization

Evidence of success in the 50 comes from deliberately broad training or data from the deployment itself. Aurora generalized to bandwidths more than 20 times above its training range, though not to the latency noise described above. Genet trains preferentially on environments where the RL policy lags rule-based baselines, and its curriculum-trained policies beat regularly trained RL and rule-based baselines, including in real environments [60]. OnRL adapts online from user sessions, and RL-CC generalized to unseen data-center scenarios in simulation. Evidence of difficulty is also present in the set: Fang et al. document a gap between simulated and real reward, and Iroko's agents overfit to the training topology.

Recent independent evaluations fall outside the 50, probably because they have had little time to gather citations, but they bear directly on **RQ2**. Giacomoni and Parisis find that public RL controllers acquire bandwidth quickly and resist random loss, but cause excess loss, are unfair in almost all setups and lag when conditions change [105]. Mazilu et al. report similar results against CUBIC and BBRv3 [106]. Ghermezian et al. deployed an Aurora-style controller on a 10 Gbit/s wide-area path, where it reached 0.5% mean utilization while CUBIC and BBR reached up to 9.9 and 9.7 Gbit/s [107]. A reader who sees only the most-cited work would miss these results.

3.6 Recommendations

From this evidence we suggest that authors state the training environment and the highest evaluation level, test at least one factor outside the training distribution (including delay noise and competing traffic), compare against CUBIC, BBR and at least one public learned controller while measuring harm to competing flows, report results over several seeds with failed runs included, report inference cost and where the model runs, and release code and traces. Of these, the 50 satisfy generalization testing in 16% of design studies and code release in 16%.

4 Deployment: barriers, LLM-designed controllers, and open problems

4.1 What the deployed systems have in common

Table 3 lists the 5 studies among the 50 that report production use or tests with real users. Each is run by an organization that controls the sender or the switches and can therefore monitor, test and roll back. Each

Table 3: Studies among the 50 reporting production use or tests with real users.

System	Setting	Learned component	Classic in loop	Reported scale
TCP-RL [69]	Web search	initial window (group-based RL); algorithm selection (deep RL)	yes	top global search engine for more than a year; 23% shorter transmission for short flows
Concerto [70]	Video calls	bitrate policy imitated from logs	no	production video-telephony application
OnRL [85]	Mobile live video	per-user online RL, aggregated model	yes	543 h, 151 users; 14.2% fewer stalls
Loki [75]	Real-time video	RL fused with a rule-based model	yes	commercial system; 101 million sessions
ACC [95]	Data center	ECN threshold per switch (multi-agent RL)	yes	production fabrics, 300 machines

optimizes a narrow service metric: transfer time, stalls or storage IOPS. Four of the five keep a classic algorithm or safeguard in the loop. The exception, Concerto, was learned from logged sessions and required no online exploration. None is a general-purpose controller for Internet traffic.

4.2 Barriers

Generalization. Deployments have avoided this problem by training on their own traffic (OnRL, Loki, Concerto) or by limiting what a wrong decision can do (ACC, TCP-RL).

Safety and worst-case behavior. Verification proves properties of trained policies or finds counterexamples [102, 103], while the deployed systems rely on runtime safeguards such as OnRL's. Outside the ranked set, Canopy adds verifier feedback to training to improve worst-case property satisfaction [108].

Fairness and coexistence. Astraea and Owl report good fairness among flows, and Mutant reports negligible harm to competing flows. Only 12 of the 43 designs evaluate fairness in any form, and independent evaluation finds public controllers unfair in almost all setups [105].

Overhead and datapath integration. Orca and Spine separate timescales. LiteFlow [109] puts neural inference in a kernel fast path, tunes in a user-space slow path, and reports 44.4% more goodput than prior deployment approaches. Hardware is stricter: outside the ranked set, NVIDIA's follow-up to RL-CC distilled its policy into decision trees, cutting inference time 500-fold to fit a NIC [110].

Interpretability. Metis offers one approach to explaining deep-learning networking systems. We did not find, among the 50, a study in which operators used such explanations.

Objectives. Every RL design hard-codes a trade-off. MOCC and AUTO condition on preferences, and deployments avoid the question by optimizing a single service metric.

Evaluation and reproducibility. Shared platforms such as Pantheon and OpenNetLab exist, but only 16% of the design studies in the 50 release code and 16% test outside their training range.

Training cost and lifecycle. Offline learning (Sage, Concerto) removes online exploration, and OnRL's aggregated agents show one way to learn in the field. The material we read rarely discusses when to retrain, how to detect drift or how to roll out a model.

4.3 LLM-designed controllers

None of the 50 uses a large language model. This line of work began around 2024, so it has had little time to gather citations. We summarize it here from our wider search, as context outside the ranked set. PolicySmith generates heuristics that integrate into the Linux kernel [111], and He et al. combine generated algorithms with an emulation pipeline and report improvements of up to 27% over BBR [112]. CCA Reimagined confines the LLM to selected phases and stresses when to invoke it [113]. NetRL fine-tunes an LLM policy and reports 24.1% higher network utility than Orca [114]. Trailblazer invokes compact LLMs only in complex conditions and outperformed an optimized industrial policy in a large A/B test on Douyin [115]. These run-time designs stay off the per-packet path, the same division of labor Orca and Spine adopted. Generated code can be reviewed and versioned like any heuristic, but a generator can overfit to the emulator that scores it.

4.4 Open problems

The evidence above suggests the following directions.

1. Benchmarks with explicit training and test distributions along named factors, reported per factor.
2. Coexistence tests against CUBIC and BBR that measure harm to competing flows [14].
3. Simulators and emulators validated against real wireless and satellite paths, as Pantheon did for wired paths.
4. Runtime assurance as a studied design component, including when to switch to a fallback and what guarantees result.
5. Offline learning from production logs with shared datasets and off-policy evaluation [116].
6. Learned components built for kernel, QUIC, NIC or switch datapaths from the start.
7. LLM-generated controllers evaluated on held-out conditions and real networks.
8. Model lifecycle: updates, drift detection, rollback and fleet-wide versioning.

4.5 Threats to validity

Citation selection. Raw citation counts favor older papers and prominent venues, and they do not measure quality. Only 4 of the 50 first appeared in 2023 or later, while 70% of the other studies did. Results here should not be generalized to the literature as a whole. Counts come from one index on one date, so another index would produce a different list near the cutoff, and the choice of 50 is itself a practical limit, set by time.

Evidence depth. Only 19 of the 50 were coded from full text. For the others, items that abstracts rarely report are likely undercounted. We did not re-validate results reported by the studies. A spot check during this report found one coding error in our extraction table, on the evaluation of Metis, and other errors may remain.

Search. We substituted OpenAlex for direct queries of IEEE Xplore, the ACM Digital Library and Scopus, because their programmatic interfaces require institutional keys. OpenAlex can miss records whose abstracts it lacks. The update search is a further deviation that we added after reading the first results.

Selection and coding. Our protocol called for both authors to screen every record independently. In this version, screening and extraction were carried out in a single pass with tool support, including keyword filtering, automatic mining of evidence and an AI assistant, and they have not yet been duplicated independently. Borderline inclusion decisions should be read as single-reviewer judgments.

5 Conclusion

We reviewed the 50 most-cited of 362 eligible studies on RL for congestion control.
RQ1 (Design). The studies observe short histories of throughput, delay and loss, act mainly on the window or the rate, and optimize a weighted reward. 15 of 43 designs keep a classic algorithm or safeguard in the deployed loop.
RQ2 (Testing). 43% of the studies with a known evaluation level reach a real network, more than among the other studies. Only 16% of designs test outside their training range, and 16% release code.
RQ3 (Deployment). All 5 deployed systems are run by operators who own the endpoints or switches and optimize a narrow service metric. Four keep a classic component in the loop, and the fifth learned from logs. Generalization, fairness against the deployed base and model lifecycle remain open.
Citation ranking gave us a tractable sample of influential work, but it also left out the recent independent evaluations and nearly all of the post-2023 literature. Extending the review to the full set of eligible studies is the most direct way to address that limit.

References

[1] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols*. ACM, Aug. 1988, pp. 314–329. [Online]. Available: http://dx.doi.org/10.1145/52324.52356

[2] D.-M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989. [Online]. Available: http://dx.doi.org/10.1016/0169-7552(89)90019-6

[3] A. Narayanan, E. Ramadan, J. Carpenter, Q. Liu, Y. Liu, F. Qian, and Z.-L. Zhang, "A First Look at Commercial 5G Performance on Smartphones," in *Proceedings of The Web Conference 2020*. ACM, Apr. 2020, pp. 894–905. [Online]. Available: http://dx.doi.org/10.1145/3366423.3380169

[4] N. Mohan, A. E. Ferguson, H. Cech, R. Bose, P. R. Renatin, M. K. Marina, and J. Ott, "A Multifaceted Look at Starlink Performance," in *Proceedings of the ACM Web Conference 2024*. ACM, May 2024, pp. 2723–2734. [Online]. Available: http://dx.doi.org/10.1145/3589334.3645328

[5] J. Gettys and K. Nichols, "Bufferbloat: dark buffers in the internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, Jan. 2012. [Online]. Available: http://dx.doi.org/10.1145/2063176.2063196

[6] H. Jiang, Y. Wang, K. Lee, and I. Rhee, "Tackling bufferbloat in 3G/4G networks," in *Proceedings of the 2012 Internet Measurement Conference*. ACM, Nov. 2012, pp. 329–342. [Online]. Available: http://dx.doi.org/10.1145/2398776.2398810

[7] H. Haile, K.-J. Grinnemo, S. Ferlin, P. Hurtig, and A. Brunstrom, "End-to-end congestion control approaches for high throughput and low delay in 4G/5G cellular networks," *Computer Networks*, vol. 186, p. 107692, 2021.

[8] Z. Zhou, S. Huang, D. Dong, Y. Bai, and L. Xiao, "End-to-end congestion control in datacenter networks: a survey," *Frontiers of Computer Science*, vol. 20, no. 5, p. 2005501, 2026.

[9] K. Winstein and H. Balakrishnan, "TCP ex machina: computer-generated congestion control," in *Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM*. ACM, Aug. 2013, pp. 123–134. [Online]. Available: http://dx.doi.org/10.1145/2486001.2486020

[10] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 395–408.

[11] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "PCC Vivace: Online-Learning Congestion Control," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018, pp. 343–356.

[12] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "A Deep Reinforcement Learning Perspective on Internet Congestion Control," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 3050–3059.

[13] R. Jain, D.-M. Chiu, and W. R. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984.

[14] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Beyond Jain's Fairness Index: Setting the Bar For The Deployment of Congestion Control Algorithms," in *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2019, pp. 17–24. [Online]. Available: http://dx.doi.org/10.1145/3365609.3365855

[15] K. Ramakrishnan, S. Floyd, and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," RFC 3168, IETF, 2001.

[16] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, IETF, 2009.

[17] S. Ha, I. Rhee, and L. Xu, "CUBIC: a new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008. [Online]. Available: http://dx.doi.org/10.1145/1400097.1400105

[18] L. Brakmo and L. Peterson, "TCP Vegas: end to end congestion avoidance on a global Internet," *IEEE Journal on Selected Areas in Communications*, vol. 13, no. 8, pp. 1465–1480, 1995. [Online]. Available: http://dx.doi.org/10.1109/49.464716

[19] V. Arun and H. Balakrishnan, "Copa: Practical Delay-Based Congestion Control for the Internet," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018, pp. 329–342.

[20] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, no. 5, pp. 20–53, Oct. 2016. [Online]. Available: http://dx.doi.org/10.1145/3012426.3022184

[21] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993. [Online]. Available: http://dx.doi.org/10.1109/90.251892

[22] K. Nichols and V. Jacobson, "Controlling queue delay," *Communications of the ACM*, vol. 55, no. 7, pp. 42–50, 2012. [Online]. Available: http://dx.doi.org/10.1145/2209249.2209264

[23] R. Pan, P. Natarajan, F. Baker, and G. White, "Proportional Integral Controller Enhanced (PIE): A Lightweight Control Scheme to Address the Bufferbloat Problem," RFC 8033, IETF, 2017.

[24] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 conference*. ACM, Aug. 2010, pp. 63–74. [Online]. Available: http://dx.doi.org/10.1145/1851182.1851192

[25] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[26] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1-2, pp. 99–134, May 1998. [Online]. Available: http://dx.doi.org/10.1016/S0004-3702(98)00023-X

[27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: http://dx.doi.org/10.1038/nature14236

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[29] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous Methods for Deep Reinforcement Learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, vol. 48, 2016, pp. 1928–1937.

[30] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous Control with Deep Reinforcement Learning," in *International Conference on Learning Representations (ICLR)*, 2016.

[31] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1587–1596.

[32] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu, "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1407–1416.

[33] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems," arXiv:2005.01643, 2020.

[34] S. Ross, G. J. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15, 2011, pp. 627–635.

[35] K. Zhang, Z. Yang, and T. Başar, *Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms*. Springer International Publishing, 2021, pp. 321–384. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-60990-0_12

[36] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers, "A practical guide to multi-objective reinforcement learning and planning," *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, Apr. 2022. [Online]. Available: http://dx.doi.org/10.1007/s10458-022-09552-y

[37] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, p. n71, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1136/bmj.n71

[38] B. Kitchenham and S. Charters, "Guidelines for Performing Systematic Literature Reviews in Software Engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[39] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, May 2014, pp. 1–10. [Online]. Available: http://dx.doi.org/10.1145/2601248.2601268

[40] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, 2018. [Online]. Available: http://dx.doi.org/10.1186/s13174-018-0087-2

[41] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019. [Online]. Available: http://dx.doi.org/10.1109/COMST.2019.2916583

[42] F. Tang, B. Mao, Y. Kawamoto, and N. Kato, "Survey on Machine Learning for Intelligent End-to-End Communication Toward 6G: From Network Access, Routing to Traffic Control and Streaming Adaption," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1578–1598, 2021.

[43] L. Zhang, Y. Cui, M. Wang, Z. Yang, and Y. Jiang, "Machine Learning for Internet Congestion Control: Techniques and Challenges," *IEEE Internet Computing*, vol. 23, no. 5, pp. 59–64, 2019.

[44] T. Zhang and S. Mao, "Machine Learning for End-to-End Congestion Control," *IEEE Communications Magazine*, vol. 58, no. 6, pp. 52–57, 2020.

[45] W. Wei, H. Gu, and B. Li, "Congestion Control: A Renaissance with Machine Learning," *IEEE Network*, vol. 35, no. 4, pp. 262–269, 2021.

[46] H. Jiang, Q. Li, Y. Jiang, G. Shen, R. Sinnott, C. Tian, and M. Xu, "When machine learning meets congestion control: A survey and comparison," *Computer Networks*, vol. 192, p. 108033, 2021.

[47] M. J. Lakshmi and M. BabuArrama, "Methods of Congestion Control in Wired Networks with Reinforcement Learning – A Review," in *2022 2nd International Conference on Innovative Practices in Technology and Management (ICIPTM)*. IEEE, Feb. 2022, pp. 128–136.

[48] M. Maliha, G. Habibi, and M. Atiquzzaman, "A Survey on Congestion Control and Scheduling for Multipath TCP: Machine Learning vs Classical Approaches," in *Proceedings of the 18th Conference on Computer Science and Intelligence Systems*, vol. 35. IEEE, 2023, pp. 49–61.

[49] M. P. Toopchinezhad and M. Ahmadi, "Machine learning approaches for active queue management: A survey, taxonomy, and future directions," *Computer Networks*, vol. 262, p. 111174, May 2025.

[50] M. E. Haque, "Fairness-Aware Congestion Control- From Heuristic Protocols to Learning-Based Frameworks," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 6s, pp. 735–743, 2025.

[51] H. Li, Y. Liu, S. Zhang, and Z. Yang, "A Survey of Deep Reinforcement Learning for Network Congestion Control," in *2025 8th International Conference on Communication Engineering and Technology (ICCET)*. IEEE, May 2025, pp. 1–6.

[52] E. Abedini, M. Nickray, and A. Jalaly Bidgoly, "Dynamic Network Congestion Control: A Reinforcement Learning-based Perspective Review," *International Journal of Engineering*, vol. 40, no. 3, pp. 892–904, 2027.

[53] C. L. Vielhaus, C. v. Lengerke, V. Latzko, J. Rischke, M. Reisslein, and F. H. P. Fitzek, "Evaluating Transport Layer Congestion Control Algorithms: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 278–340, 2026.

[54] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: The Training Ground for Internet Congestion-Control Research," in *2018 USENIX Annual Technical Conference (USENIX ATC)*, 2018.

[55] V. Sivakumar, O. Delalleau, T. Rocktäschel, A. H. Miller, H. Küttler, N. Nardelli, M. Rabbat, J. Pineau, and S. Riedel, "MVFST-RL: An Asynchronous RL Framework for Congestion Control with Delayed Actions," arXiv:1910.04054, 2019.

[56] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 632–647.

[57] S. Emara, B. Li, and Y. Chen, "Eagle: Refining Congestion Control by Learning from the Experts," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*. IEEE, 2020, pp. 676–685.

[58] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto, "Owl: Congestion Control with Partially Invisible Networks via Reinforcement Learning," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. IEEE, May 2021, pp. 1–10.

[59] Y. Ma, H. Tian, X. Liao, J. Zhang, W. Wang, K. Chen, and X. Jin, "Multi-objective congestion control," in *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, Mar. 2022, pp. 218–235.

[60] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang, "Genet: automatic curriculum generation for learning adaptation in networking," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 397–413.

[61] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control," in *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 2023, pp. 255–274.

[62] H. Tian, X. Liao, C. Zeng, D. Sun, J. Zhang, and K. Chen, "Efficient DRL-Based Congestion Control With Ultra-Low Overhead," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 1888–1903, 2024.

[63] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards Fair and Efficient Learning-based Congestion Control," in *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, Apr. 2024, pp. 99–114.

[64] L. Pappone, A. Sacco, and F. Esposito, "Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.

[65] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive Congestion Control with Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2019.

[66] Y. Kong, H. Zang, and X. Ma, "Improving TCP Congestion Control with Machine Intelligence," in *Proceedings of the 2018 Workshop on Network Meets AI & ML - NetAI'18*. ACM Press, 2018, pp. 60–66.

[67] K. Xiao, S. Mao, and J. K. Tugnait, "TCP-Drinc: Smart Congestion Control Based on Deep Reinforcement Learning," *IEEE Access*, vol. 7, pp. 11 892–11 904, 2019.

[68] Z. Du, J. Zheng, H. Yu, H. Zhang, and G. Chen, "Libra: A Congestion Control Framework for Diverse Application Preferences and Network Conditions," *IEEE Transactions on Networking*, vol. 33, no. 2, pp. 510–525, Apr. 2025.

[69] X. Nie, Y. Zhao, Z. Li, G. Chen, K. Sui, J. Zhang, Z. Ye, and D. Pei, "Dynamic TCP Initial Windows and Congestion Control Schemes Through Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1231–1247, 2019.

[70] A. Zhou, H. Zhang, G. Su, L. Wu, R. Ma, Z. Meng, X. Zhang, X. Xie, H. Ma, and X. Chen, "Learning to Coordinate Video Codec with Transport Protocol for Mobile Video Telephony," in *The 25th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2019, pp. 1–16.

[71] Z. Xu, J. Tang, C. Yin, Y. Wang, and G. Xue, "Experience-Driven Congestion Control: When Multi-Path TCP Meets Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1325–1336, 2019.

[72] S. R. Pokhrel and A. Walid, "Learning to Harness Bandwidth With Multipath Congestion Control and Scheduling," *IEEE Transactions on Mobile Computing*, vol. 22, no. 2, pp. 996–1009, Feb. 2023.

[73] T. Huang, R.-X. Zhang, C. Zhou, and L. Sun, "QARC: Video Quality Aware Rate Control for Real-Time Video Streaming based on Deep Reinforcement Learning," in *Proceedings of the 26th ACM international conference on Multimedia*. ACM, Oct. 2018, pp. 1208–1216.

[74] J. Fang, M. Ellis, B. Li, S. Liu, Y. Hosseinkashi, M. Revow, A. Sadovnikov, Z. Liu, P. Cheng, S. Ashok, D. Zhao, R. Cutler, Y. Lu, and J. Gehrke, "Reinforcement learning for bandwidth estimation and congestion control in real-time communications," arXiv:1912.02222, 2019.

[75] H. Zhang, A. Zhou, Y. Hu, C. Li, G. Wang, X. Zhang, H. Ma, L. Wu, A. Chen, and C. Wu, "Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based

models," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2021, pp. 775–788.

[76] B. Wang, Y. Zhang, S. Qian, Z. Pan, and Y. Xie, "A Hybrid Receiver-side Congestion Control Scheme for Web Real-time Communication," in *Proceedings of the 12th ACM Multimedia Systems Conference*. ACM, 2021, pp. 332–338.

[77] A. Bentaleb, M. N. Akcay, M. Lim, A. C. Begen, and R. Zimmermann, "BoB: Bandwidth Prediction for Real-Time Communications Using Heuristic and Reinforcement Learning," *IEEE Transactions on Multimedia*, vol. 25, pp. 6930–6945, 2023.

[78] J. Yang, Y. Chen, K. Xue, J. Han, J. Li, D. S. L. Wei, Q. Sun, and J. Lu, "IEACC: An Intelligent Edge-Aided Congestion Control Scheme for Named Data Networking With Deep Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4932–4947, Dec. 2022.

[79] K. Lei, Y. Liang, and W. Li, "Congestion Control in SDN-Based Networks via Multi-Task Deep Reinforcement Learning," *IEEE Network*, vol. 34, no. 4, pp. 28–34, 2020.

[80] E. Sert, C. Sonmez, S. Baghaee, and E. Uysal-Biyikoglu, "Optimizing age of information on real-life TCP/IP connections through reinforcement learning," in *2018 26th Signal Processing and Communications Applications Conference (SIU)*. IEEE, May 2018, pp. 1–4.

[81] Y. Su, L. Huang, and C. Feng, "QRED: A Q-Learning-based Active Queue Management Scheme," Journal of Internet Technology, 2018.

[82] C. A. Gomez, X. Wang, and A. Shami, "Intelligent Active Queue Management Using Explicit Congestion Notification," in *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, Dec. 2019, pp. 1–6.

[83] H. Ma, D. Xu, Y. Dai, and Q. Dong, "An intelligent scheme for congestion control: When active queue management meets deep reinforcement learning," *Computer Networks*, vol. 200, p. 108515, Dec. 2021.

[84] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Wanna Make Your TCP Scheme Great for Cellular Networks? Let Machines Do It for You!" *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 265–279, Jan. 2021.

[85] H. Zhang, A. Zhou, J. Lu, R. Ma, Y. Hu, C. Li, X. Zhang, H. Ma, and X. Chen, "OnRL: improving mobile video telephony via online reinforcement learning," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. ACM, 2020, pp. 1–14.

[86] W. Li, H. Zhang, S. Gao, C. Xue, X. Wang, and S. Lu, "SmartCC: A Reinforcement Learning Approach for Multipath TCP Congestion Control in Heterogeneous Networks," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 11, pp. 2621–2633, Nov. 2019.

[87] B. He, J. Wang, Q. Qi, H. Sun, J. Liao, C. Du, X. Yang, and Z. Han, "DeepCC: Multi-Agent Deep Reinforcement Learning Congestion Control for Multi-Path TCP Based on Self-Attention," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4770–4788, Dec. 2021.

[88] R. Xie, X. Jia, and K. Wu, "Adaptive Online Decision Method for Initial Congestion Window in 5G Mobile Edge Computing Using Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 2, pp. 389–403, Feb. 2020.

[89] L. Cui, Z. Yuan, Z. Ming, and S. Yang, "Improving the Congestion Control Performance for Mobile Networks in High-Speed Railway via Deep Reinforcement Learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 6, pp. 5864–5875, 2020.

[90] W. Li, F. Zhou, W. Meleis, and K. Chowdhury, "Learning-Based and Data-Driven TCP Design for Memory-Constrained IoT," in *2016 International Conference on Distributed Computing in Sensor Systems (DCOSS)*. IEEE, May 2016, pp. 199–205.

[91] P. K. Donta, S. N. Srirama, T. Amgoth, and C. S. R. Annavarapu, "iCoCoA: intelligent congestion control algorithm for CoAP using deep reinforcement learning," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 3, pp. 2951–2966, Jan. 2023.

[92] F. Naeem, G. Srivastava, and M. Tariq, "A Software Defined Network Based Fuzzy Normalized Neural Adaptive Multipath Congestion Control for the Internet of Things," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2155–2164, Oct. 2020.

[93] M. Kim, M. Jaseemuddin, and A. Anpalagan, "Deep Reinforcement Learning Based Active Queue Management for IoT Networks," *Journal of Network and Systems Management*, vol. 29, no. 3, Apr. 2021.

[94] X. Li, F. Tang, J. Liu, L. T. Yang, L. Fu, and L. Chen, "AUTO: Adaptive Congestion Control Based on Multi-Objective Reinforcement Learning for the Satellite-Ground Integrated Network," in *2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021.

[95] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, and W. Deng, "ACC: automatic ECN tuning for high-speed datacenter networks," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 384–397.

[96] C. Tessler, Y. Shpigelman, G. Dalal, A. Mandelbaum, D. Haritan Kazakov, B. Fuhrer, G. Chechik, and S. Mannor, "Reinforcement Learning for Datacenter Congestion Control," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 2, pp. 43–46, Jan. 2022.

[97] F. Ruffy, M. Przystupa, and I. Beschastnikh, "Iroko: A Framework to Prototype Reinforcement Learning for Data Center Traffic Control," arXiv:1812.09975, 2018.

[98] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*. ACM, Oct. 2010, pp. 1–6. [Online]. Available: http://dx.doi.org/10.1145/1868447.1868466

[99] R. Netravali, A. Sivaraman, S. Das, A. Goyal, K. Winstein, J. Mickens, and H. Balakrishnan, "Mahimahi: Accurate Record-and-Replay for HTTP," in *2015 USENIX Annual Technical Conference (USENIX ATC)*, 2015, pp. 417–429.

[100] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018. [Online]. Available: http://dx.doi.org/10.1609/aaai.v32i1.11694

[101] J. Eo, Z. Niu, W. Cheng, F. Y. Yan, R. Gao, J. Kardhashi, S. Inglis, M. Revow, B.-G. Chun, P. Cheng, and Y. Xiong, "OpenNetLab: Open Platform for RL-based Congestion Control for Real-Time Communications," in *Proceedings of the 6th Asia-Pacific Workshop on Networking*. ACM, 2022, pp. 70–75.

[102] Y. Kazak, C. Barrett, G. Katz, and M. Schapira, "Verifying Deep-RL-Driven Systems," in *Proceedings of the 2019 Workshop on Network Meets AI & ML - NetAI'19*. ACM Press, 2019, pp. 83–89.

[103] T. Eliyahu, Y. Kazak, G. Katz, and M. Schapira, "Verifying learning-augmented systems," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 305–318.

[104] Z. Meng, M. Wang, J. Bai, M. Xu, H. Mao, and H. Hu, "Interpreting Deep Learning-Based Networking Systems," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 154–171.

[105] L. Giacomoni and G. Parisis, "Reinforcement Learning-based Congestion Control: A Systematic Evaluation of Fairness, Efficiency and Responsiveness," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. IEEE, May 2024, pp. 1451–1460.

[106] M. Mazilu, L. Giacomoni, and G. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv, 2025.

[107] A. Ghermezian, K. Karpov, D. Kachan, V. Kirova, and E. Siemens, "PPO-Driven Reinforcement Learning Congestion Control Under High- BDP Wide-Area Deployment: A Scaling Analysis," 2026. [Online]. Available: https://opendata.uni-halle.de//handle/1981185920/125740

[108] C. Yang, D. Saxena, R. Dwivedula, K. Mahajan, S. Chaudhuri, and A. Akella, "Canopy: Property-Driven Learning for Congestion Control," arXiv:2412.10915, 2024.

[109] J. Zhang, C. Zeng, H. Zhang, S. Hu, and K. Chen, "LiteFlow: towards high-performance adaptive neural networks for kernel datapath," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 414–427.

[110] B. Fuhrer, Y. Shpigelman, C. Tessler, S. Mannor, G. Chechik, E. Zahavi, and G. Dalal, "Implementing Reinforcement Learning Datacenter Congestion Control in NVIDIA NICs," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, May 2023, pp. 331–343.

[111] R. Dwivedula, D. Saxena, A. Akella, S. Chaudhuri, and D. Kim, "Man-Made Heuristics Are Dead. Long Live Code Generators!" in *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2025, pp. 51–60.

[112] Z. He, A. Gottipati, L. Qiu, Y. Yang, and F. Y. Yan, "Congestion Control System Optimization with Large Language Models," arXiv:2508.16074, 2025.

[113] X. Qin, Y. Wang, and L. Shangguan, "CCA Reimagined: An Exploratory Study of Large Language Models for Congestion Control," arXiv:2604.03857, 2026.

[114] L. Peng, X. Xie, Y. Bao, and Y. Cui, "NetRL: Powering Network Decisions via NetGRPO-Driven Reinforcement Learning," arXiv:2610.16226, 2026.

[115] D. Wu, L. Kang, Z. Wang, F. Wang, W. Zhang, C. Sun, X. Tao, W. Yang, L. Zhang, W. Zhu, P. Cui, and Z. Wang, "Grounding Large Language Models as Generalizable Policies in Network Control," arXiv:2512.11839, 2025.

[116] J. Fu, M. Norouzi, O. Nachum, G. Tucker, Z. Wang, A. Novikov, M. Yang, M. R. Zhang, Y. Chen, A. Kumar, C. Paduraru, S. Levine, and T. L. Paine, "Benchmarks for Deep Off-Policy Evaluation," in *International Conference on Learning Representations (ICLR)*, 2021.

A Search strings and sources

The database and citation searches were run on 31 October 2026, and the update search on 9 November 2026, for publication years 2016–2026.

OpenAlex. Works endpoint, Boolean search over titles and abstracts, page paging, 200 records per page:

```
filter = title_and_abstract.search:("reinforcement learning" OR
  "imitation learning") AND ("congestion control" OR
  "congestion window" OR ECN), publication_year:2016-2026
```

arXiv. Because the API's phrase operator under-retrieves, we retrieved on single terms and applied the protocol's exact phrases locally (hyphens treated as spaces; "ECN" matched as a whole word):

```
search_query = (ti:reinforcement OR abs:reinforcement OR ti:imitation
  OR abs:imitation) AND (ti:congestion OR abs:congestion OR ti:ECN
  OR abs:ECN) AND submittedDate:[201601010000 TO 202612312359]
```

USENIX NSDI and ATC. The technical-sessions page of every NSDI (2016–2026) and USENIX ATC (2016–2025) was scraped for titles and abstracts, and the protocol query was applied locally.

Citation searching. For forward searching, we looked for works published between 2016 and 2026 that cited at least one of ten seed reports and used "congestion" in the title or abstract. The seeds were Aurora (arXiv 2018 and ICML 2019), Orca, Pantheon/Indigo, MVFST-RL, Sage, QTCP, Eagle, TCP-RL, and DRL-CC for MPTCP. For backward searching, we checked the reference lists of sixteen recent included reports with detailed related-work sections, including Sage, Orca, and Astraea.

Update search with broadened terms.

```
OpenAlex (2016-2026, titles and abstracts):
  ("reinforcement learning" OR "imitation learning" OR "Q-learning"
   OR "Q learning" OR "deep Q-network" OR DQN OR DDPG OR "actor-critic"
   OR "policy gradient" OR "multi-armed bandit" OR "contextual bandit"
   OR "offline RL" OR "deep RL" OR DRL OR "learned congestion"
   OR "learning-based congestion") AND
  ("congestion control" OR "congestion window" OR ECN OR TCP OR QUIC
   OR "active queue management" OR AQM OR "bandwidth estimation"
   OR "sending rate" OR "pacing rate" OR "congestion avoidance"
   OR "congestion management" OR "network congestion" OR RDMA
   OR incast OR "initial window" OR WebRTC)
arXiv (2016-2026):
  (ti:reinforcement OR abs:reinforcement OR ti:imitation OR abs:imitation
   OR abs:bandit OR ti:bandit OR abs:"Q-learning" OR abs:DQN OR abs:DRL
   OR abs:"actor-critic" OR abs:"offline RL" OR abs:"deep RL") AND
  (abs:TCP OR ti:TCP OR abs:QUIC OR ti:QUIC OR abs:congestion
   OR ti:congestion OR abs:AQM OR abs:"queue management"
   OR abs:"bandwidth estimation" OR abs:WebRTC OR abs:RDMA OR abs:incast)
```

B The 50 reviewed studies

Table 4: The 50 most-cited eligible studies by Semantic Scholar count on 9 November 2026. Setting: WAN/Gen = Internet or generic wired, Wless = wireless or cellular, IoT = IoT, ad hoc or sensor, DC = data center, Sat = satellite. Level: highest evaluation environment (NR = not reported; – = formal analysis). Source of coding: FT = full text, AB = abstract and metadata, KN = reviewers' knowledge.

#	Cites	Study	Year	Setting	Method	Level	Src
1	497	Aurora [12]	2019	WAN/Gen	PPO	Emu	FT
2	337	Orca [56]	2020	WAN/Gen	TD3 (distributed actors)	Real	KN
3	313	Pantheon (Indigo) [54]	2018	WAN/Gen	imitation learning from emulator oracles (Indigo, LSTM)	Real	FT
4	201	QTCP [65]	2018	WAN/Gen	Q-learning + generalized Kanerva coding	NR	AB
5	156	Genet [60]	2022	WAN/Gen	curriculum learning promoting environments where RL lags rule-based baselines	Real	FT
6	153	TCP-RL [69]	2019	WAN/Gen	group-based RL (IW) + deep RL (CCA selection among 14)	Real	AB
7	147	DRL-CC (MPTCP) [71]	2019	WAN/Gen	deterministic policy gradient actor-critic + LSTM	Emu	AB
8	145	OnRL [85]	2020	Wless	online RL: per-user agents, aggregated model, safeguards against risky exploration	Real	AB
9	141	Metis [104]	2020	WAN/Gen	decision-tree and hypergraph interpretation of DL networking systems	Sim	FT
10	139	Owl [58]	2021	WAN/Gen	DQN in a Linux kernel module	Emu	FT
11	114	QARC [73]	2018	WAN/Gen	A3C + video-quality prediction network	Sim	FT
12	113	SmartCC [86]	2019	Wless	Q-learning + hierarchical tile coding (asynchronous)	NR	AB
13	108	Sage [61]	2023	WAN/Gen	offline RL learning from a pool of existing heuristics' trajectories	Real	AB
14	101	Concerto [70]	2019	WAN/Gen	custom imitation learning on >1M logged sessions	Real	AB
15	100	whiRL 2.0 [103]	2021	WAN/Gen	DNN verification + model checking of DRL policies	–	AB
16	95	Loki [75]	2021	WAN/Gen	RL model fused with a reverse-engineered rule-based model (dual attention), trained online	Real	AB
17	91	RL-TCP [66]	2018	WAN/Gen	Q-learning	Sim	AB
18	83	Verily [102]	2019	WAN/Gen	formal verification of DRL policies	–	AB
19	74	TCP-Drinc [67]	2019	WAN/Gen	DQN with CNN+LSTM	Sim	FT
20	64	Eagle [57]	2020	WAN/Gen	DRL with LSTM guided by BBR as expert	Emu	KN
21	61	DQL-MPTCP [72]	2021	WAN/Gen	deep Q-learning with policy gradients, LSTM, prioritized replay	Emu	FT
22	61	MOCC [59]	2022	WAN/Gen	PPO with preference sub-network + transfer learning (user- and kernel-space)	Real	FT
23	58	DeepCC (Abbasloo et al.) [84]	2020	Wless	DDPG plug-in capping the cwnd of existing TCP	Real	FT
24	54	DeepCC (He et al., MPTCP) [87]	2021	Wless	multi-agent DRL with self-attention	NR	AB
25	53	SDN actor-critic MPTCP for IoT [92]	2020	IoT	actor-critic with fuzzy normalized NN	Sim	AB
26	52	DRL initial window for 5G MEC [88]	2019	Wless	DRL (+supervised refinement)	Sim	AB
27	52	RL BWE for RTC (Microsoft) [74]	2019	WAN/Gen	PPO (actor-critic)	Real	FT
28	51	RL-CC (Tessler et al.) [96]	2022	DC	policy gradient with analytic reward derivative (multi-agent, partially observable)	Sim	FT
29	49	Spine [62]	2023	WAN/Gen	hierarchical: light CC executor + coarse-grained DRL sub-policy generator	Real	AB
30	46	Libra [68]	2024	WAN/Gen	classic and RL candidates compared in exploration/evaluation stages	Real	AB
31	45	DQN AQM for fog/edge IoT [93]	2021	IoT	DQN	Sim	AB
32	45	iCoCoA [91]	2023	IoT	DRL	NR	AB
33	44	Astraea [63]	2024	WAN/Gen	multi-agent DRL with a faithful multi-flow training environment	Real	FT
34	42	MVFST-RL [55]	2019	WAN/Gen	IMPALA (asynchronous actor-learner, V-trace off-policy correction) with delayed actions	Emu	FT
35	39	AUTO [94]	2021	Sat	multi-objective RL + preference-adaptation model	Real	FT

#	Cites	Study	Year	Setting	Method	Level	Src
36	37	Iroko [97]	2018	DC	PPO; REINFORCE; DDPG	Emu	FT
37	37	Multi-task DRL CC (SDN) [79]	2020	WAN/Gen	multi-task DRL	NR	AB
38	37	ACC [95]	2021	DC	multi-agent DRL per switch (offline + online training)	Real	AB
39	36	Q-learning TCP for memory-constrained IoT [90]	2016	IoT	Q-learning with function approximation	Sim	AB
40	36	OpenNetLab [101]	2022	WAN/Gen	platform for training/evaluating RL RTC CC	Real	AB
41	35	DRL age-of-information over TCP/IP [80]	2018	WAN/Gen	deep RL	Emu	AB
42	35	BoB [77]	2022	WAN/Gen	PPO + heuristic warm start	Emu	FT
43	32	Mutant [64]	2025	WAN/Gen	online contextual bandit (LinUCB) over a pool of existing protocols	Real	FT
44	31	QRED [81]	2018	WAN/Gen	Q-learning tuning RED maxp	Sim	AB
45	31	LiteFlow [109]	2022	WAN/Gen	kernel fast path for NN inference + userspace slow path for tuning	Emu	AB
46	30	HRCC [76]	2021	WAN/Gen	RL agent sets a gain on the heuristic (GCC-like) bandwidth estimate	Sim	AB
47	29	Intelligent AQM with ECN [82]	2019	WAN/Gen	Q-learning (+ANN congestion predictor)	Emu	FT
48	28	IEACC [78]	2022	WAN/Gen	DRL with edge-aided congestion degrees	Sim	AB
49	27	Hd-TCP [89]	2020	Wless	DRL	Sim	AB
50	27	AQM meets DRL [83]	2021	WAN/Gen	DRL	NR	AB